MERN Stack Practicals – Index & Code Manual

Syllabus order: Practical 1 to Practical 10. Each practical begins on a new page. Follow the short setup steps, then paste each code block into the exact file path shown immediately above it.

No.	Practical	Source PDF
1	Environment Setup and JavaScript Basics	Pract1.docx.pdf
2	Core JavaScript Concepts for MERN	Pract2.docx.pdf
3	React Introduction and JSX	Pract3_Corrected.pdf
4	React Components, Props, and State	Practical_4_Corrected.pdf
5	React Routing and Data Handling	Pract5.docx.pdf
6	Node.js and Express Fundamentals	Pract6.docx.pdf
7	MongoDB Basics	Pract7_Corrected.pdf
8	API Creation and Integration	Practical 8.pdf
9	User Data Management	Practical 9.pdf
10	Full Stack Execution and Validation	Practical 10.pdf

General notes

• Use VS Code for editing files. Use the integrated terminal or PowerShell for commands. Do not paste terminal commands into JavaScript files.

• For React practicals using Create React App, run `npm start` from the React project root. The default address is http://localhost:3000. Some original screenshots show Vite's 5173 address; follow the commands in the practical, not that screenshot address.

• For Practicals 8–10, keep the Express server terminal running while starting React in a second terminal. MongoDB must be running locally.

Practical 1 · Environment Setup and JavaScript Basics

Source PDF: Pract1.docx.pdf

Steps

1. Install Node.js and VS Code. Open VS Code and create/open folder Pract1.
2. Create the .js files shown in the next pages inside Pract1. Paste each JavaScript block into its named file.
3. Open VS Code · Terminal · New Terminal. Run the file you want to test.

PowerShell / VS Code terminal

```
node -v
npm -v
mkdir Pract1
cd Pract1
code .
```

Run the files

```
node app.js
node basics.js
node functions.js
node operators.js
node loops.js
```

Paste source code into the file paths shown on the following practical pages.

Practical 1 – Environment Setup and JavaScript Basics

Source PDF: Pract1.docx.pdf

Files/folders: create a folder named Pract1. Open it in VS Code. Create the JavaScript files below. Run each file from the VS Code terminal with `node filename.js`.

1. Install Node.js (LTS) and Visual Studio Code. Open a new terminal and check: node -v, then npm -v.

2. In VS Code, create/open folder Pract1. Create app.js for the first Node.js example.

3. Paste the relevant code into the named file. Save it, then run node app.js in the terminal.

4. For the remaining exercises, use the separate filenames shown below and run each with node filename.js.

Path: Pract1/app.js

File: app.js

```
console.log("Hello, Node.js!");
```

Path: Pract1/basics.js

File: basics.js

```
// Console Output
console.log("JavaScript Basics");

// Variables
let name = "Pooja";
let age = 30;

// Display variables
console.log("Name:", name);
console.log("Age:", age);

// Arithmetic Operations
let a = 15;
let b = 5;

console.log("Addition:", a + b);
console.log("Subtraction:", a - b);
console.log("Multiplication:", a * b);
console.log("Division:", a / b);
console.log("Modulus:", a % b);
```

Path: Pract1/functions.js

File: functions.js

```
let num1 = 10;
let num2 = 20;

function add(x, y) {
    return x + y;
}

let result = add(num1, num2);
console.log("Sum =", result);

function greet(name) {
    return `Welcome ${name}!`;
}

console.log(greet("Pooja"));
```

Path: Pract1/operators.js

File: operators.js

```
let x = 12;
let y = 4;

console.log("Addition:", x + y);
console.log("Greater Than:", x > y);
console.log("Equal To:", x == y);
console.log("Logical AND:", (x > 5 && y < 10));

// If-Else Statement
let marks = 75;

if (marks >= 40) {
    console.log("Pass");
} else {
    console.log("Fail");
}

// Switch Statement
let day = 3;

switch (day) {
```

```
    case 1:
        console.log("Monday");
        break;
    case 2:
        console.log("Tuesday");
        break;
    case 3:
        console.log("Wednesday");
        break;
    default:
        console.log("Invalid Day");
}
```

Path: Pract1/loops.js

File: loops.js

```
// For loop
for (let i = 1; i <= 5; i++) {
    console.log(i);
}

// While loop
let count = 1;

while (count <= 5) {
    console.log(count);
    count++;
}

// Template Literals
let student = "Pooja";
let score = 90;

console.log(`Student Name: ${student}`);
console.log(`Score: ${score}`);
```

PowerShell note: the source PDF shows `Set-ExecutionPolicy RemoteSigned`. This is not JavaScript and is not needed to run these Node.js files. Do not paste it into a .js file.

Practical 2 · Core JavaScript Concepts for MERN

Source PDF: Pract2.docx.pdf

Steps

1. Open VS Code and create/open folder Pract2.
2. Create app.js inside Pract2. Paste the complete Practical 2 code from the next page into app.js and save.
3. Open Terminal · New Terminal in VS Code.

PowerShell / VS Code terminal

```
node -v
npm -v
mkdir Pract2
cd Pract2
code .
node app.js
```

Paste source code into the file paths shown on the following practical pages.

Practical 2 – Core JavaScript Concepts for MERN

Source PDF: Pract2.docx.pdf

Folder: Pract2. Open this folder in VS Code. Create app.js, paste the complete code below, save, and run `node app.js` in the terminal.

1. Open VS Code → File → Open Folder → Pract2.

2. Create app.js in the Pract2 folder.

3. Paste the code below and save (Ctrl+S).

4. Open Terminal → New Terminal and run node app.js.

Path: Pract2/app.js

File: app.js

```
// a. Functions and parameters
function greet(name) {
    return "Hello, " + name + "!";
}

console.log(greet("Alice"));
console.log(greet("John"));

// b. Arrays and objects
let fruits = ["Apple", "Banana", "Mango"];

console.log(fruits[0]);
fruits.push("Orange");
console.log(fruits);

// c. Arrow functions
const add = (a, b) => {
    return a + b;
};

console.log(add(10, 20));

// d. Array methods: map, filter, reduce
let numbers = [10, 20, 30, 40, 50];

let doubled = numbers.map(num => num * 2);
let greaterThan25 = numbers.filter(num => num > 25);
let total = numbers.reduce((sum, num) => sum + num, 0);

console.log("Original:", numbers);
console.log("Doubled:", doubled);
console.log("Greater than 25:", greaterThan25);
console.log("Sum:", total);

// e. Object manipulation
let employee = {
    id: 101,
    name: "Amit",
    department: "IT",
    salary: 50000
};

employee.salary = 60000;
employee.city = "Mumbai";
delete employee.department;

console.log(employee);

// f. Functions, arrays and objects together
const students = [
    { name: "Alice", marks: 85 },
    { name: "Bob", marks: 72 },
    { name: "Charlie", marks: 90 },
    { name: "David", marks: 65 }
];

const averageMarks = (students) => {
    const total = students.reduce(
        (sum, student) => sum + student.marks, 0
    );
    return total / students.length;
};

const passedStudents = (students) => {
    return students.filter(student => student.marks >= 75);
};

console.log("Average Marks:", averageMarks(students));
```

```
console.log("Passed Students:");

passedStudents(students).forEach(student => {
    console.log(`${student.name} - ${student.marks}`);
});
```

Practical 3 · React Introduction and JSX

Source PDF: Pract3_Corrected.pdf

Steps

1. Create the React project. Open its folder in VS Code.
2. For each exercise, replace src/App.js with that exercise's code. Create App.css, Greeting.jsx or DateTime.jsx when specified above the code.
3. Keep the terminal running. Open the browser at http://localhost:3000.

PowerShell / VS Code terminal

```
npx create-react-app pract3app
cd pract3app
code .
npm start
```

Run later

```
cd pract3app
npm start
```

Note: This project uses Create React App, so open http://localhost:3000. Some source screenshots show 5173, which is Vite's default port and does not match the CRA commands.

Paste source code into the file paths shown on the following practical pages.

Practical 3 – React Introduction and JSX

Source PDF: Pract3_Corrected.pdf

Project folder: pract3app. Use VS Code. These exercises are separate App.js examples: for each part, replace the contents of src/App.js with that part's code. Keep the other named files in src.

1. Open terminal and run: npx create-react-app pract3app

2. Run: cd pract3app, then npm start.

3. In VS Code open pract3app. Open src/App.js and replace its default code with the required part below.

4. For CSS, also create/replace src/App.css. For the component exercise, create Greeting.jsx. For the clock, create DateTime.jsx.

5. The browser opens at http://localhost:3000. Keep npm start running.

Path: pract3app/src/App.js

File: App.js (Part a)

```
function App() {
    return (
        <div>
            <h1>Welcome to MERN Stack Development</h1>
        </div>
    );
}

export default App;
```

Path: pract3app/src/App.js

File: App.js (Part b)

```
function App() {
    const name = "Pooja";
    const course = "MERN Stack Development";

    return (
        <div>
            <h2>Hello, {name}</h2>
            <p>Welcome to {course}</p>
        </div>
    );
}

export default App;
```

Path: pract3app/src/App.js

File: App.js (Part c)

```
function App() {
    const num1 = 25;
    const num2 = 15;

    function calculateSum(a, b) {
        return a + b;
    }

    return (
        <div>
            <h2>Sum Calculation</h2>
            <p>Number 1: {num1}</p>
            <p>Number 2: {num2}</p>
            <p>Sum: {calculateSum(num1, num2)}</p>
        </div>
    );
}

export default App;
```

Path: pract3app/src/App.css

File: App.css (Part d)

```
.container {
    text-align: center;
    background-color: lightblue;
    padding: 20px;
    margin: 20px;
    border-radius: 10px;
}

h1 {
    color: navy;
}

p {
```

```
    color: darkgreen;
    font-size: 18px;
}
```

Path: pract3app/src/App.js

File: App.js (Part d)

```
import "./App.css";

function App() {
    return (
        <div className="container">
            <h1>React CSS Example</h1>
            <p>This component is styled using CSS.</p>
        </div>
    );
}

export default App;
```

Path: pract3app/src/Greeting.jsx

File: Greeting.jsx (Part e)

```
function Greeting() {
    return <h2>Hello from Greeting Component!</h2>;
}

export default Greeting;
```

Path: pract3app/src/App.js

File: App.js (Part e)

```
import Greeting from "./Greeting";

function App() {
    return (
        <div>
            <h1>Main App Component</h1>
            <Greeting />
        </div>
    );
}

export default App;
```

Path: pract3app/src/DateTime.jsx

File: DateTime.jsx (Part f)

```
import { useState, useEffect } from "react";

function DateTime() {
    const [currentDate, setCurrentDate] = useState(new Date());

    useEffect(() => {
        const timer = setInterval(() => {
            setCurrentDate(new Date());
        }, 1000);

        return () => clearInterval(timer);
    }, []);

    return (
        <div>
            <h2>Current Date and Time</h2>
            <p>Date: {currentDate.toLocaleDateString()}</p>
            <p>Time: {currentDate.toLocaleTimeString()}</p>
        </div>
    );
}

export default DateTime;
```

Path: pract3app/src/App.js

File: App.js (Part f)

```
import DateTime from "./DateTime";

function App() {
    return (
        <div>
            <h1>Date and Time Example</h1>
            <DateTime />
        </div>
    );
}

export default App;
```

Practical 4 · React Components, Props, and State

Source PDF: Practical_4_Corrected.pdf

Steps

1. Create the React project and open it in VS Code.
2. Each part is a separate exercise: replace src/App.js with the relevant App.js code. For the Props part, also create src/Student.js.
3. Keep npm start running and check http://localhost:3000.

PowerShell / VS Code terminal

```
npx create-react-app pract4app
cd pract4app
code .
npm start
```

Run later

```
cd pract4app
npm start
```

Paste source code into the file paths shown on the following practical pages.

Practical 4 – React Components, Props, and State

Source PDF: Practical_4_Corrected.pdf

Project folder: pract4app. Create it with Create React App. Each part is a separate exercise. Replace src/App.js with the relevant App.js code before testing that part. For Part B, also create Student.js.

1. Run: npx create-react-app pract4app

2. Run: cd pract4app, then npm start.

3. Open pract4app in VS Code → src.

4. For each part, paste its code into src/App.js. For Part B, create src/Student.js as well.

5. Save and check the browser at http://localhost:3000.

Path: pract4app/src/App.js

File: App.js (Part A)

```
// Component 1
function Header() {
    return <h1>Welcome to React</h1>;
}

// Component 2
function Student() {
    return <h2>Student Information</h2>;
}

// Component 3
function Course() {
    return <p>Course: MERN Stack Development</p>;
}

// Main Component
function App() {
    return (
        <div>
            <Header />
            <Student />
            <Course />
        </div>
    );
}

export default App;
```

Path: pract4app/src/Student.js

File: Student.js (Part B)

```
function Student(props) {
    return (
        <div>
            <h2>Student Details</h2>
            <p>Name: {props.name}</p>
            <p>Course: {props.course}</p>
        </div>
    );
}

export default Student;
```

Path: pract4app/src/App.js

File: App.js (Part B)

```
import Student from "./Student";

function App() {
    return (
        <div>
            <h1>React Props Example</h1>
            <Student
                name="Pooja"
                course="MERN Stack Development"
            />
        </div>
    );
}

export default App;
```

Path: pract4app/src/App.js

File: App.js (Part C – Counter)

```
import React, { useState } from "react";
```

```
function App() {
    const [count, setCount] = useState(0);

    return (
        <div>
            <h1>Counter</h1>
            <h2>Count: {count}</h2>

            <button onClick={() => setCount(count + 1)}>
                Increase
            </button>

            <button onClick={() => setCount(count - 1)}>
                Decrease
            </button>

            <button onClick={() => setCount(0)}>
                Reset
            </button>
        </div>
    );
}

export default App;
```

Path: pract4app/src/App.js

File: App.js (Part D – Form)

```
import React, { useState } from "react";

function App() {
    const [name, setName] = useState("");

    return (
        <div>
            <h1>Student Form</h1>
            <input
                type="text"
                placeholder="Enter your name"
                value={name}
                onChange={(e) => setName(e.target.value)}
            />
            <h2>Hello, {name}</h2>
        </div>
    );
}

export default App;
```

Path: pract4app/src/App.js

File: App.js (Part E – List)

```
import React, { useState } from "react";

function App() {
    const [students] = useState([
        "Pooja",
        "Rahul",
        "Sneha",
        "Amit"
    ]);

    return (
        <div>
            <h1>Student List</h1>
            <ul>
                {students.map((student, index) => (
                    <li key={index}>
                        {student}
                    </li>
                ))}
            </ul>
        </div>
    );
}

export default App;
```

Path: pract4app/src/App.js

File: App.js (Part F – To-Do List)

```
import React, { useState } from "react";

function App() {
    const [task, setTask] = useState("");
    const [todos, setTodos] = useState([]);

    // Add task
```

```
    const addTask = () => {
        if (task !== "") {
            setTodos([...todos, task]);
            setTask("");
        }
    };

    // Delete task
    const deleteTask = (index) => {
        setTodos(todos.filter((todo, i) => i !== index));
    };

    return (
        <div>
            <h1>My To-Do List</h1>

            <input
                type="text"
                placeholder="Enter a task"
                value={task}
                onChange={(e) => setTask(e.target.value)}
            />

            <button onClick={addTask}>
                Add Task
            </button>

            <h2>Tasks</h2>
            <ul>
                {todos.map((todo, index) => (
                    <li key={index}>
                        {todo}
                        <button onClick={() => deleteTask(index)}>
                            Delete
                        </button>
                    </li>
                ))}
            </ul>
        </div>
    );
}

export default App;
```

Practical 5 · React Routing and Data Handling

Source PDF: Pract5.docx.pdf

Steps

1. Create the React project. Install React Router and Axios before pasting the code.
2. In VS Code create src/pages. Put Home.js, About.js and Users.js inside src/pages; put App.js inside src.
3. Start React and open http://localhost:3000.

PowerShell / VS Code terminal

```
npx create-react-app react-routing-app
cd react-routing-app
npm install react-router-dom axios
code .
npm start
```

Run later

```
cd react-routing-app
npm start
```

Paste source code into the file paths shown on the following practical pages.

Practical 5 – React Routing and Data Handling

Source PDF: Pract5.docx.pdf

Project folder: react-routing-app. Use VS Code for files and terminal for commands. Create the pages folder inside src. Paste each block into its named file.

1. Run: npx create-react-app react-routing-app

2. Run: cd react-routing-app

3. Run: npm install react-router-dom axios

4. In VS Code open react-routing-app/src. Create folder pages.

5. Create App.js in src and Home.js, About.js, Users.js inside src/pages.

6. Run npm start from react-routing-app. Browser: http://localhost:3000.

Path: react-routing-app/src/App.js

File: App.js

```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";

import Home from "./pages/Home";
import About from "./pages/About";
import Users from "./pages/Users";

function App() {
    return (
        <BrowserRouter>
            <div>
                <nav>
                    <ul style={{ display: "flex", gap: "20px" }}>
                        <li>
                            <Link to="/">Home</Link>
                        </li>
                        <li>
                            <Link to="/about">About</Link>
                        </li>
                        <li>
                            <Link to="/users">Users</Link>
                        </li>
                    </ul>
                </nav>

                <Routes>
                    <Route path="/" element={<Home />} />
                    <Route path="/about" element={<About />} />
                    <Route path="/users" element={<Users />} />
                </Routes>
            </div>
        </BrowserRouter>
    );
}

export default App;
```

Path: react-routing-app/src/pages/Home.js

File: Home.js

```
function Home() {
    return (
        <div>
            <h1>Home Page</h1>
            <p>Welcome to React Router Demo</p>
        </div>
    );
}

export default Home;
```

Path: react-routing-app/src/pages/About.js

File: About.js

```
function About() {
    return (
        <div>
            <h1>About Page</h1>
            <p>This application demonstrates routing and API fetching.</p>
        </div>
    );
}

export default About;
```

Path: react-routing-app/src/pages/Users.js

File: Users.js

```
import axios from "axios";
import { useEffect, useState } from "react";

function Users() {
    const [users, setUsers] = useState([]);
    const [search, setSearch] = useState("");

    useEffect(() => {
        axios
            .get("https://jsonplaceholder.typicode.com/users")
            .then((response) => {
                setUsers(response.data);
            })
            .catch((error) => {
                console.log(error);
            });
    }, []);

    const filteredUsers = users.filter((user) =>
        user.name.toLowerCase().includes(search.toLowerCase())
    );

    return (
        <div>
            <h1>Users List</h1>

            <input
                type="text"
                placeholder="Search User..."
                value={search}
                onChange={(e) => setSearch(e.target.value)}
            />

            <table border="1" cellPadding="10">
                <thead>
                    <tr>
                        <th>ID</th>
                        <th>Name</th>
                        <th>Email</th>
                        <th>City</th>
                    </tr>
                </thead>

                <tbody>
                    {filteredUsers.map((user) => (
                        <tr key={user.id}>
                            <td>{user.id}</td>
                            <td>{user.name}</td>
                            <td>{user.email}</td>
                            <td>{user.address.city}</td>
                        </tr>
                    ))}
                </tbody>
            </table>
        </div>
    );
}

export default Users;
```

Practical 6 · Node.js and Express Fundamentals

Source PDF: Pract6.docx.pdf

Steps

1. Create/open folder Pract6 in VS Code. Create server.js in this folder and paste the Express code from the next page.
2. Install Express, then start the server. Keep the terminal open while testing the routes in a browser.

PowerShell / VS Code terminal

```
mkdir Pract6
cd Pract6
code .
npm init -y
npm install express
node server.js
```

Test in browser

```
http://localhost:3000
http://localhost:3000/about
http://localhost:3000/contact
http://localhost:3000/student
```

Paste source code into the file paths shown on the following practical pages.

Practical 6 – Node.js and Express Fundamentals

Source PDF: Pract6.docx.pdf

Folder: Pract6. Use VS Code to create server.js. Use terminal for package installation and server execution. Open each route in a browser to test it.

1. Create/open folder Pract6 in VS Code.

2. Open Terminal in this folder and run: npm init -y

3. Run: npm install express

4. Create server.js in the Pract6 folder and paste the code below.

5. Run: node server.js. Keep the terminal running.

6. In a browser open http://localhost:3000, /about, /contact and /student.

Path: Pract6/server.js

File: server.js

```
const express = require("express");

const app = express();
const PORT = 3000;

// Home Route
app.get("/", (req, res) => {
    res.send("Welcome to Home Page");
});

// About Route
app.get("/about", (req, res) => {
    res.send("About Express Application");
});

// Contact Route
app.get("/contact", (req, res) => {
    res.send("Contact Us");
});

// JSON Route
app.get("/student", (req, res) => {
    res.json({
        id: 1,
        name: "Rahul",
        age: 21,
        course: "Node.js"
    });
});

app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
});
```

Practical 7 · MongoDB Basics

Source PDF: Pract7_Corrected.pdf

Steps

1. Open MongoDB Compass and connect to mongodb://localhost:27017. Create database emp and collection e1.
2. For shell commands, open PowerShell and run mongosh. Run the MongoDB commands only inside mongosh, not in a .js file.
3. For the Mongoose connection example, create app.js in Pract7 and install mongoose.

Start MongoDB (Windows service, if installed)

```
Open PowerShell as Administrator:
net start MongoDB
```

MongoDB Shell

```
mongosh
use emp
db.e1.find({})
```

Mongoose connection · VS Code terminal

```
mkdir Pract7
cd Pract7
npm init -y
npm install mongoose
node app.js
```

Paste source code into the file paths shown on the following practical pages.

Practical 7 – MongoDB Basics

Source PDF: Pract7_Corrected.pdf

Use MongoDB Compass for the database/collection and visual CRUD steps. Use MongoDB Shell (mongosh) for the commands. Use VS Code only for the Mongoose connection file.

1. Open MongoDB Compass and connect to mongodb://localhost:27017.

2. Click Create Database. Database name: emp. Collection name: e1. Click Create Database.

3. Open emp → e1 → Add Data → Insert Document. Insert the sample document below; repeat with other employee documents if needed.

4. Use the Compass Update and Delete buttons as shown in the source practical, or run the shell commands below in mongosh.

5. For Node.js connection, create folder Pract7, run npm init -y and npm install mongoose. Create app.js and run node app.js.

Path: MongoDB Compass → emp → e1 → Add Data → Insert Document

File: Document JSON

```
{
    "emp_nm": "A",
    "emp_desg": "clerk",
    "emp_sal": 34000
}
```

Path: mongosh terminal

File: MongoDB Shell commands

```
use emp

// Insert one employee
db.e1.insertOne({
    "emp_nm": "D",
    "emp_desg": "manager",
    "emp_sal": 70000
})

// View all records
db.e1.find({}, {})

// Find employee C
db.e1.find({"emp_nm": "C"}, {})

// Show selected fields (MongoDB includes _id by default)
db.e1.find({}, {"emp_nm": 1, "emp_desg": 1})

// Update all managers' salaries
db.e1.updateMany(
    {"emp_desg": "manager"},
    {"$set": {"emp_sal": 80000}}
)

// Delete records with salary 80000
db.e1.deleteMany({"emp_sal": 80000})
```

Path: Pract7/app.js

File: app.js

```
const mongoose = require("mongoose");

// Connection URI to Mongoose
const uri = "mongodb://127.0.0.1:27017/emp";

// Make database connection (asynchronously)
mongoose.connect(uri)
    .then((result) => {
        console.log("Successful connection!!.....");
    })
    .catch((error) => console.log(error));
```

The source practical demonstrates database creation and CRUD through Compass screenshots, shell commands, and a Mongoose connection. It does not provide a Mongoose schema/model CRUD application for this practical.

Practical 8 · API Creation and Integration

Source PDF: Practical 8.pdf

Steps

1. Create the React project. In VS Code create server.js inside pract8app/src and replace src/App.js with the React code.
2. Start MongoDB first. Use two terminals: one for the backend and one for React.
3. Keep both terminals running. Frontend: http://localhost:3000; API: http://localhost:5000/students.

PowerShell / VS Code terminal · setup

```
npx create-react-app pract8app
cd pract8app
npm install express mongoose cors
code .
```

Terminal 1 · backend

```
cd pract8app\src
node server.js
```

Terminal 2 · React frontend

```
cd pract8app
npm start
```

Paste source code into the file paths shown on the following practical pages.

Practical 8 – API Creation and Integration

Source PDF: Practical 8.pdf

Project folder: Pract8/pract8app. The source places server.js inside src. Create React App files (App.jsx and server.js) are both inside src. Use two terminals: one for the backend and one for React.

1. In terminal create React project: npx create-react-app pract8app, then cd pract8app.

2. Run: npm install express mongoose cors

3. In VS Code create server.js inside pract8app/src and replace src/App.js with the App code below.

4. Start MongoDB. Terminal 1: cd pract8app/src, then node server.js. Keep it running.

5. Terminal 2: cd pract8app, then npm start. Open http://localhost:3000.

Path: Pract8/pract8app/src/server.js

File: server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();

app.use(cors());
app.use(express.json());

// Connect to MongoDB
mongoose
    .connect("mongodb://127.0.0.1:27017/studentDB")
    .then(() => console.log("MongoDB Connected"))
    .catch((err) => console.log(err));

// Schema
const studentSchema = new mongoose.Schema({
    name: String,
    age: Number
});

// Model
const Student = mongoose.model("Student", studentSchema);

// GET API - retrieve data
app.get("/students", async (req, res) => {
    const students = await Student.find();
    res.json(students);
});

// POST API - receive and store data
app.post("/students", async (req, res) => {
    const student = new Student({
        name: req.body.name,
        age: req.body.age
    });

    await student.save();

    res.json({
        message: "Student saved successfully",
        student: student
    });
});

// Start server
app.listen(5000, () => {
    console.log("Server running on port 5000");
});
```

Path: Pract8/pract8app/src/App.js

File: App.js

```
import { useEffect, useState } from "react";

function App() {
    const [students, setStudents] = useState([]);
    const [name, setName] = useState("");
    const [age, setAge] = useState("");

    // GET data from API
    const getStudents = async () => {
        const response = await fetch("http://localhost:5000/students");
        const data = await response.json();
        setStudents(data);
    };
```

```
    // POST data to API
    const addStudent = async (e) => {
        e.preventDefault();

        await fetch("http://localhost:5000/students", {
            method: "POST",
            headers: {
                "Content-Type": "application/json"
            },
            body: JSON.stringify({
                name: name,
                age: Number(age)
            })
        });

        setName("");
        setAge("");

        // Refresh displayed data
        getStudents();
    };

    // Get students when page loads
    useEffect(() => {
        getStudents();
    }, []);

    return (
        <div>
            <h1>Student Management</h1>

            <form onSubmit={addStudent}>
                <input
                    type="text"
                    placeholder="Enter name"
                    value={name}
                    onChange={(e) => setName(e.target.value)}
                />

                <input
                    type="number"
                    placeholder="Enter age"
                    value={age}
                    onChange={(e) => setAge(e.target.value)}
                />

                <button type="submit">Add Student</button>
            </form>

            <h2>Students</h2>

            {students.map((student) => (
                <p key={student._id}>
                    {student.name} - {student.age} years
                </p>
            ))}
        </div>
    );
}

export default App;
```

Practical 9 · User Data Management

Source PDF: Practical 9.pdf

Steps

1. Create the React project. In VS Code create server.js and App.js inside pract9app/src.
2. Start MongoDB first. Use two terminals: backend in Terminal 1 and React in Terminal 2.
3. Use the form to add, edit and delete users.

PowerShell / VS Code terminal · setup

```
npx create-react-app pract9app
cd pract9app
npm install express mongoose cors
code .
```

Terminal 1 · backend

```
cd pract9app\src
node server.js
```

Terminal 2 · React frontend

```
cd pract9app
npm start
```

Postman API tests

```
GET    http://localhost:5000/users
POST   http://localhost:5000/users
PUT    http://localhost:5000/users/RECORD_ID
DELETE http://localhost:5000/users/RECORD_ID
For POST/PUT: Body ? raw ? JSON. Replace RECORD_ID with a real MongoDB _id.
```

Paste source code into the file paths shown on the following practical pages.

Practical 9 – User Data Management

Source PDF: Practical 9.pdf

Project folder: Pract9/pract9app. Put server.js and App.js in pract9app/src as shown in the source structure. Keep MongoDB running. Use two terminals for backend and frontend.

1. Create project: npx create-react-app pract9app, then cd pract9app.

2. Run: npm install express mongoose cors

3. In VS Code create pract9app/src/server.js and replace pract9app/src/App.js with the code below.

4. Start MongoDB. Terminal 1: cd pract9app/src, then node server.js.

5. Terminal 2: cd pract9app, then npm start. Open http://localhost:3000.

6. Use the form to add users. Click Edit to load a record into the form, then Update User. Click Delete to remove a record.

Path: Pract9/pract9app/src/server.js

File: server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();

app.use(cors());
app.use(express.json());

// Connect MongoDB
mongoose
    .connect("mongodb://127.0.0.1:27017/userDB")
    .then(() => console.log("MongoDB Connected"))
    .catch((err) => console.log(err));

// User Schema
const userSchema = new mongoose.Schema({
    name: String,
    email: String,
    age: Number
});

const User = mongoose.model("User", userSchema);

// ADD USER
app.post("/users", async (req, res) => {
    const user = new User(req.body);
    await user.save();
    res.json(user);
});

// GET USERS
app.get("/users", async (req, res) => {
    const users = await User.find();
    res.json(users);
});

// UPDATE USER
app.put("/users/:id", async (req, res) => {
    const user = await User.findByIdAndUpdate(
        req.params.id,
        req.body,
        { new: true }
    );
    res.json(user);
});

// DELETE USER
app.delete("/users/:id", async (req, res) => {
    await User.findByIdAndDelete(req.params.id);
    res.json({ message: "User deleted" });
});

// Start server
app.listen(5000, () => {
    console.log("Server running on port 5000");
});
```

Path: Pract9/pract9app/src/App.js

File: App.js

```
import { useEffect, useState } from "react";

function App() {
```

```
const [users, setUsers] = useState([]);
const [name, setName] = useState("");
const [email, setEmail] = useState("");
const [age, setAge] = useState("");
const [editId, setEditId] = useState(null);

// GET USERS
const getUsers = async () => {
    const response = await fetch("http://localhost:5000/users");
    const data = await response.json();
    setUsers(data);
};

// ADD / UPDATE USER
const saveUser = async (e) => {
    e.preventDefault();

    const userData = {
        name: name,
        email: email,
        age: Number(age)
    };

    if (editId) {
        // UPDATE
        await fetch(`http://localhost:5000/users/${editId}`, {
            method: "PUT",
            headers: {
                "Content-Type": "application/json"
            },
            body: JSON.stringify(userData)
        });

        setEditId(null);
    } else {
        // ADD
        await fetch("http://localhost:5000/users", {
            method: "POST",
            headers: {
                "Content-Type": "application/json"
            },
            body: JSON.stringify(userData)
        });
    }

    clearForm();
    getUsers();
};

// DELETE USER
const deleteUser = async (id) => {
    await fetch(`http://localhost:5000/users/${id}`, {
        method: "DELETE"
    });
    getUsers();
};

// EDIT USER
const editUser = (user) => {
    setName(user.name);
    setEmail(user.email);
    setAge(user.age);
    setEditId(user._id);
};

// CLEAR FORM
const clearForm = () => {
    setName("");
    setEmail("");
    setAge("");
};

// Load users when page opens
useEffect(() => {
    getUsers();
}, []);

return (
    <div>
        <h1>User Data Management</h1>

        <form onSubmit={saveUser}>
            <input
                type="text"
                placeholder="Name"
                value={name}
```

```
                    onChange={(e) => setName(e.target.value)}
                />

                <input
                    type="email"
                    placeholder="Email"
                    value={email}
                    onChange={(e) => setEmail(e.target.value)}
                />

                <input
                    type="number"
                    placeholder="Age"
                    value={age}
                    onChange={(e) => setAge(e.target.value)}
                />

                <button type="submit">
                    {editId ? "Update User" : "Add User"}
                </button>
            </form>

            <h2>User List</h2>

            {users.map((user) => (
                <div key={user._id}>
                    <p>
                        <b>{user.name}</b> - {user.email} - {user.age} years
                    </p>

                    <button onClick={() => editUser(user)}>
                        Edit
                    </button>

                    <button onClick={() => deleteUser(user._id)}>
                        Delete
                    </button>
                </div>
            ))}
        </div>
    );
}

export default App;
```

Practical 10 · Full Stack Execution and Validation

Source PDF: Practical 10.pdf

Steps

1. Create the React project. In VS Code create server.js and App.js inside pract10app/src.
2. Start MongoDB first. Use two terminals: backend in Terminal 1 and React in Terminal 2.
3. Test validation, add, edit and delete in the React page. Also test all APIs in Postman.

PowerShell / VS Code terminal · setup

```
npx create-react-app pract10app
cd pract10app
npm install express mongoose cors
code .
```

Terminal 1 · backend

```
cd pract10app\src
node server.js
```

Terminal 2 · React frontend

```
cd pract10app
npm start
```

Postman API tests

```
GET    http://localhost:5000/users
POST   http://localhost:5000/users
PUT    http://localhost:5000/users/RECORD_ID
DELETE http://localhost:5000/users/RECORD_ID
For POST/PUT: Body ? raw ? JSON. Replace RECORD_ID with a real MongoDB _id.
```

Paste source code into the file paths shown on the following practical pages.

Practical 10 – Full Stack Execution and Validation

Source PDF: Practical 10.pdf

Project folder: Pract10/pract10app. Put server.js and App.js in pract10app/src as shown in the source structure. Keep MongoDB running. Use two terminals.

1. Create project: npx create-react-app pract10app, then cd pract10app.

2. Run: npm install express mongoose cors

3. In VS Code create pract10app/src/server.js and replace pract10app/src/App.js with the code below.

4. Start MongoDB. Terminal 1: cd pract10app/src, then node server.js.

5. Terminal 2: cd pract10app, then npm start. Open http://localhost:3000.

6. Test add, edit, delete and validation in the React page. Use Postman to test GET/POST/PUT/DELETE at http://localhost:5000/users.

Path: Pract10/pract10app/src/server.js

File: server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();

app.use(cors());
app.use(express.json());

// MongoDB connection
mongoose
    .connect("mongodb://127.0.0.1:27017/userDB")
    .then(() => console.log("MongoDB Connected"))
    .catch((err) => console.log(err));

// User Schema
const userSchema = new mongoose.Schema({
    name: String,
    email: String,
    age: Number
});

const User = mongoose.model("User", userSchema);

// GET - Display users
app.get("/users", async (req, res) => {
    const users = await User.find();
    res.json(users);
});

// POST - Add user
app.post("/users", async (req, res) => {
    const user = new User(req.body);
    await user.save();
    res.json(user);
});

// PUT - Update user
app.put("/users/:id", async (req, res) => {
    const user = await User.findByIdAndUpdate(
        req.params.id,
        req.body,
        { new: true }
    );
    res.json(user);
});

// DELETE - Delete user
app.delete("/users/:id", async (req, res) => {
    await User.findByIdAndDelete(req.params.id);
    res.json({ message: "User deleted successfully" });
});

// Start server
app.listen(5000, () => {
    console.log("Server running on port 5000");
});
```

Path: Pract10/pract10app/src/App.js

File: App.js

```
import { useEffect, useState } from "react";
```

```
function App() {
    const [users, setUsers] = useState([]);
    const [name, setName] = useState("");
    const [email, setEmail] = useState("");
    const [age, setAge] = useState("");
    const [editId, setEditId] = useState(null);
    const [error, setError] = useState("");

    // Get users
    const getUsers = async () => {
        const response = await fetch("http://localhost:5000/users");
        const data = await response.json();
        setUsers(data);
    };

    // Form validation
    const validateForm = () => {
        if (name.trim() === "") {
            setError("Name is required");
            return false;
        }

        if (!email.includes("@")) {
            setError("Enter a valid email");
            return false;
        }

        if (age <= 0) {
            setError("Age must be greater than 0");
            return false;
        }

        setError("");
        return true;
    };

    // Add or Update
    const saveUser = async (e) => {
        e.preventDefault();

        if (!validateForm()) {
            return;
        }

        const userData = {
            name: name,
            email: email,
            age: Number(age)
        };

        if (editId) {
            // UPDATE
            await fetch(`http://localhost:5000/users/${editId}`, {
                method: "PUT",
                headers: {
                    "Content-Type": "application/json"
                },
                body: JSON.stringify(userData)
            });

            setEditId(null);
        } else {
            // INSERT
            await fetch("http://localhost:5000/users", {
                method: "POST",
                headers: {
                    "Content-Type": "application/json"
                },
                body: JSON.stringify(userData)
            });
        }

        clearForm();

        // Refresh data dynamically
        getUsers();
    };

    // Delete
    const deleteUser = async (id) => {
        await fetch(`http://localhost:5000/users/${id}`, {
            method: "DELETE"
        });
        getUsers();
    };
```

```
    // Edit
    const editUser = (user) => {
        setName(user.name);
        setEmail(user.email);
        setAge(user.age);
        setEditId(user._id);
    };

    // Clear form
    const clearForm = () => {
        setName("");
        setEmail("");
        setAge("");
        setError("");
    };

    // Load users when page starts
    useEffect(() => {
        getUsers();
    }, []);

    return (
        <div>
            <h1>User Management</h1>

            <form onSubmit={saveUser}>
                <input
                    type="text"
                    placeholder="Name"
                    value={name}
                    onChange={(e) => setName(e.target.value)}
                />

                <br /><br />

                <input
                    type="email"
                    placeholder="Email"
                    value={email}
                    onChange={(e) => setEmail(e.target.value)}
                />

                <br /><br />

                <input
                    type="number"
                    placeholder="Age"
                    value={age}
                    onChange={(e) => setAge(e.target.value)}
                />

                <br /><br />

                <button type="submit">
                    {editId ? "Update User" : "Add User"}
                </button>

                {error && <p style={{ color: "red" }}>{error}</p>}
            </form>

            <h2>Users</h2>

            {users.map((user) => (
                <div key={user._id}>
                    <p>
                        {user.name} | {user.email} | {user.age}
                    </p>

                    <button onClick={() => editUser(user)}>
                        Edit
                    </button>

                    <button onClick={() => deleteUser(user._id)}>
                        Delete
                    </button>

                    <hr />
                </div>
            ))}
        </div>
    );
}

export default App;
```

Postman quick test: GET http://localhost:5000/users. POST the same URL with Body → raw → JSON, for example: {"name":"Asha","email":"asha@example.com","age":20}. For PUT use /users/RECORD_ID with JSON body. For DELETE use /users/RECORD_ID. Replace RECORD_ID with the MongoDB document _id.